

Plain

Twenty-nine words, one way to write each thing, and a mistake tells you *what to do about it* instead of what went wrong inside the machine.

Everything the language has is in here: all 29 words and all 24 built-in actions, with a list of the lot at the end.

Contents

1	The shape of a program
2	The six kinds of value
3	The three kinds of bracket
4	set and change — names and values
5	show — displaying something
6	How things combine — the important part
7	Arithmetic and joining
8	Comparing
9	Logic and conditions
10	Loops
11	Actions
12	Lists in detail
13	Records in detail
14	The 24 built-in actions
15	Reaching outside the program
16	Order of operations
17	Where names live
18	Errors
19	The complete word list
20	What Plain does not have yet
21	A complete program
22	Everything, in alphabetical order

Version 0.5 · Complete. Every word the language has is in this document.

Plain has 29 words and 24 built-in actions. That's the whole language — nothing to install and nothing to import.

1. The shape of a program

A program is a list of instructions, one per line, run from top to bottom.

```
set name to "Mark"
show "Hello, " + name
```

Four rules cover all of it:

- **One instruction per line.** No semicolons.
- **Indentation is yours.** Blocks are closed by the word `end`, so spacing can never change what a program does. Indent for readability or don't.
- **# starts a note.** Everything after it on that line is ignored.
- **Blank lines mean nothing.** Use them freely.

```
# This is a note. The line below is not.
set total to 0
```

2. The six kinds of value

KIND	WRITTEN AS	NOTES
Number	12, 3.5, -8	One kind only. No separate whole/decimal types.
Text	"hello"	Always double quotes.
True/false	true, false	The only things an <code>if</code> will accept.
List	[1, 2, 3]	Items count from 1.
Record	{name: "Ada"}	Named fields.
Nothing	nothing	No value yet.

There is a seventh kind you create yourself — an **action**, made with `make`. See section 9.

Inside text you can use `\n` for a new line, `\"` for a quote mark, and `\\` for a backslash.

3. The three kinds of bracket

Each pair does one job. Mixing them up is the most common early mistake, so it's worth a page of its own.

BRACKETS	JOB	EXAMPLE
[]	Make a list, or reach into one	[1, 2, 3] and xs[1]
{ }	Make a record, and look one up by name	{name: "Ada"} and counts["the"]
()	Group a calculation, and hold an action's inputs	(2 + 3) * 4 and double(5)

Two traps catch nearly everyone.

Round brackets don't make a list. `(1, 2, 3)` is an error, and Plain will offer to rewrite it as `[1, 2, 3]`.

Square brackets build a new list — they don't mark where one goes. If you already have a list, hand it over as it is:

```
set a to [1, 2]
set b to [3]

show join(a + b, "-")    # 1-2-3
show join([a + b], "-")  # wrong: a list holding one list
```

`[a + b]` makes a list with a single item in it, and that item is itself a list. `count([a + b])` is 1, while `count(a + b)` is 3.

4. `set` and `change` — names and values

`set` creates a name. `change` gives an existing one a new value. They are separate on purpose, so a typo can never quietly create a second variable.

```
set total to 0      # create
change total to 5    # alter
```

Each fails clearly if used the wrong way round:

```
set total to 0
set total to 5
```

Line 2: `total` already exists. To give it a new value, use `change total to 5`.

```
change score to 10
```

Line 1: There's nothing called `score` to change. Create it first with `set score to 10`.

```
set total to 0
change totl to total + 5
```

Line 2: There's nothing called `totl` to change. There is something called `total` — did you mean that?

That last one is the point of the whole design. In most languages it silently makes a second variable and the bug surfaces much later.

There is no `=` in Plain, so `=` and `==` can never be confused.

One name means one thing. `set` fails if the name is already in use anywhere you can see it, including the built-in actions:

```
set count to 3
```

Line 1: `count` is already the name of a built-in action.

Names start with a letter or `_` and may contain letters, digits and `_`. They are case-sensitive.

The 24 built-in action names are taken, so `set count to 0` won't work — `count` is already something. The error says which name clashed. Section 14 has the full list; `total`, `tally` and `how_many` are all free.

`change` also reaches inside lists and records:

```

set xs to [10, 20, 30]
change xs[2] to 99      # xs is now [10, 99, 30]

set p to {name: "Ada"}
change p.name to "Grace"
change p.city to "London" # adds a new field

```

`set` only ever creates a whole new name, so it never has a `.` or `[]` after it.

A NAME HOLDS A VALUE, NOT A LINK

Giving one name to another copies the value. The two are then independent:

```

set prices to [10, 20, 30]
set backup to prices

change prices[1] to 99

show prices      # [99, 20, 30]
show backup      # [10, 20, 30]

```

Numbers and text have always worked this way — `set b to a` then changing `a` never altered `b`. Lists and records work the same way, so nothing gets shared by accident.

The same is true when a value goes into an action, which is what makes an action properly sealed:

```

set data to [1, 2, 3]

make wreck(xs)
  change xs[1] to 999
  give 0
end

show wreck(data)
show data      # [1, 2, 3] — untouched

```

An action can read what you pass in and hand something back. It can't reach out and change anything.

If you want two names to match again later, say so — `change backup to prices` takes a fresh copy at that moment.

5. `show` — displaying something

```

show "Hello"
show 42
show total

```

Several values separated by commas appear on one line, with a space between:

```

set prices to [10, 20]
show "How many:", count(prices), "Total:", sum(prices)

```

```
How many: 2 Total: 30
```

`show` displays text without its quote marks. Inside a list or record, text keeps its quotes so you can see what is text and what is a number.

6. How things combine — the important part

This is the rule that makes everything else fit together:

Anywhere a value is allowed, any expression that produces a value is allowed.

An **expression** is anything that produces a value: a number, some text, a name, a list, arithmetic, a comparison, a field, a position, or an action call. A **statement** is a whole instruction: `set`, `change`, `show`, `if`, `for each`, `repeat`, `while`, `make`, `give`, `stop`.

So `count(prices)` produces a number, and can go anywhere a number can go:

```
show count(prices)           # in a show
set how_many to count(prices) # in a set
if count(prices) is more than 3 # in a test
show "Items: " + count(prices) # joined onto text
set doubled to count(prices) * 2 # in arithmetic
```

Actions can go inside other actions. Read them from the inside out:

```
show first(sort_up(prices))
# sort_up(prices) gives an ordered list
# first(...) then takes its first item
```

```
show round(sum(prices) / count(prices))
# sum gives a total, count gives how many,
# / divides them, round makes it whole
```

Nest as deeply as you like. There is no limit and no special syntax for it.

What does not combine: a statement can never go inside an expression. These are all wrong:

```
set x to show 5           # show is a statement, not a value
set x to (set y to 2)     # set is a statement, not a value
show if true              # if is a statement, not a value
```

Where each thing must produce the right kind of value:

POSITION	MUST BE
<code>if</code> and <code>while</code> tests	true or false — nothing else is accepted
<code>for each ... in</code>	a list, or text
<code>repeat ... times</code>	a number
<code>[position]</code>	a number, from 1 upwards

7. Arithmetic and joining

```
show 10 + 5    # 15
show 10 - 5    # 5
show 10 * 5    # 50
show 10 / 4    # 2.5
show 10 % 3    # 1 — the remainder after dividing
```

`+` does three jobs, depending on what is on either side:

```
show 2 + 3           # 5           number plus number
show "a" + "b"       # ab          text joined to text
show "Total: " + 30   # Total: 30  number folded into text
show [1, 2] + [3]     # [1, 2, 3]  two lists into one
```

That third line matters: joining a number onto text needs no conversion. `"Total: " + 30` simply works.

Dividing by zero is an error, not `infinity`.

Numbers are shown tidied, so `0.1 + 0.2` displays as `0.3` rather than a long trail of digits.

8. Comparing

Comparisons are words, never symbols. Typing `>` or `==` gets you an error telling you the word to use instead.

WRITTEN	MEANS
<code>is</code>	exactly the same
<code>is not</code>	different
<code>is more than</code>	bigger
<code>is less than</code>	smaller
<code>is at least</code>	bigger or the same
<code>is at most</code>	smaller or the same

```
if score is 10
if name is not "Ada"
if age is at least 18
```

`is` compares by content, so two lists with the same items match:

```
show [1, 2] is [1, 2]    # true
```

Sizes can be compared for numbers, and for text alphabetically. **Capital letters and accents are ignored when ordering**, so text sorts the way a dictionary does:

```
show "apple" is less than "Banana"  # true
show sort_up(["banana", "Apple"])   # ["Apple", "banana"]
```

Ordering ignores capitals, but `is` doesn't — `"Apple" is "apple"` is false. Matching is exact — only ordering is forgiving.

Comparing sizes of anything else is an error, with a message pointing you at `is`.

9. Logic and conditions

```
and  both must be true
or   either may be true
not  turns true into false
```

```

if age is at least 18 and name is not ""
  show "Allowed"
end

if not (score is 0)
  show "Started"
end

```

Everything must be true or false. There is no truthiness — an empty list, zero, and empty text are **not** false in Plain, they are simply not conditions:

```

if count(xs) is more than 0    # correct
if xs                          # error, with a message telling you the above

```

IF, ELSE IF, ELSE

```

if score is at least 9
  show "Publish"
else if score is at least 7
  show "Watch"
else
  show "Skip"
end

```

One `end` closes the whole chain. `else if` may repeat as many times as you like; `else` is optional and comes last.

10. Loops

FOR EACH — GO THROUGH A LIST

```

for each price in prices
  show price
end

```

Also works on text, one letter at a time:

```

for each letter in "abc"
  show letter
end

```

For a record, go through its field names:

```

for each field in keys(person)
  show field
end

```

REPEAT — A FIXED NUMBER OF TIMES

```

repeat 3 times
  show "tick"
end

```

The count must be a whole number. `repeat 2.7 times` is an error rather than a silent guess, and the message says to use `round(2.7)` if that's what you meant.

WHILE – UNTIL A TEST STOPS BEING TRUE

```
set n to 1
while n is at most 5
  show n
  change n to n + 1
end
```

STOP – LEAVE A LOOP EARLY

```
for each n in [1, 2, 3, 4]
  if n is 3
    stop
  end
  show n
end
```

```
1
2
```

`stop` works in all three loops and leaves only the innermost one.

A loop that **never finishes** is stopped automatically after about four million steps or five seconds, with a message saying so. It will not hang.

11. Actions

`make` creates an action. `give` hands a value back.

```
make double(n)
  give n * 2
end

show double(21)      # 42
```

An action sees only what you pass in. It cannot read or alter anything outside itself, so its first line lists everything it uses:

```
set tax to 0.2
make total(amount)
  give amount * (1 + tax)  # error – tax is outside
end
```

Line 3: `tax` exists outside this action, but actions can't see outside names. Pass it in instead.

```
make total(amount, tax)      # correct
  give amount * (1 + tax)
end
show total(100, 0.2)        # 120
```

Two useful consequences. Names are reusable — `n` can be an input to every action, since no two actions can see each other's. And nothing ever changes at a distance: to alter something outside, an action must give a value back and the caller must `change` it.

Actions can still call other actions, including themselves:

```

make factorial(n)
  if n is at most 1
    give 1
  end
  give n * factorial(n - 1)
end

show factorial(6)      # 720

```

Actions are made at the top level only, never inside an `if`, a loop, or another action. Actions can call each other freely, so nothing is lost by keeping them together — and you can see every action in a program without hunting through blocks.

Several inputs are separated by commas, and an action with no inputs still needs its brackets:

```

make greet()
  show "Hello"
end

greet()

```

`give` stops the action immediately, so it doubles as an early exit. An action that never reaches a `give` produces `nothing`. Calling with the wrong number of values is an error naming what it expected.

Actions are values, so they can be stored and passed on:

```

make double(n)
  give n * 2
end

make apply(fn, value)
  give fn(value)
end

show apply(double, 7)  # 14

```

12. Lists in detail

Items count from 1. The first item is `xs[1]`, matching how people already count.

```

set xs to ["a", "b", "c"]
show xs[1]      # a
show xs[3]      # c
show count(xs)  # 3
change xs[2] to "Z"  # xs is now ["a", "Z", "c"]

```

Asking for a position that doesn't exist tells you how many there are. A position must also be a whole number — `xs[2.9]` is an error, not item 2, because a fraction there is nearly always a sum that needed rounding.

A list may hold anything, including other lists and records:

```

set people to [
  {name: "Ada", born: 1815},
  {name: "Alan", born: 1912}
]

for each person in people
  show person.name + " — " + person.born
end

```

A list written across several lines needs no continuation marks, as above.

Text also takes positions, giving one letter:

```
show "hello"[1]      # h
```

13. Records in detail

```
set person to {name: "Ada", born: 1815}
show person.name      # Ada
show keys(person)     # ["name", "born"]
show count(person)    # 2
```

Reading a field that doesn't exist lists the fields it does have, and suggests a correction if your spelling was close.

Records nest, and are read with dots all the way down:

```
set config to {owner: {name: "Ada", city: "London"}}
show config.owner.city # London
```

A record's fields are fixed when you create it. `change` can alter a field but never invent one:

```
set person to {name: "Ada"}
change person.age to 36
```

Line 2: This record has no field called `age`, so there's nothing to change. It has: `name`. A record's fields are fixed when you create it.

So a misspelling is caught rather than quietly making a second field alongside the real one.

LOOKING A RECORD UP BY NAME

The dot needs a field you wrote down yourself. When the name is worked out while the program runs — counting words, grouping things, keeping a tally — use square brackets and text instead:

```
set counts to {}

for each word in split("the cat the dog the cat", " ")
  if has(counts, word)
    change counts[word] to counts[word] + 1
  else
    change counts[word] to 1
  end
end

show counts      # {the: 3, cat: 2, dog: 1}
show counts["the"] # 3
```

`change ...["key"] to ...` adds the entry if it isn't there yet. That's the one place a record can grow, and you had to type the quotes to get it, so it can't happen by accident.

Reading a name that isn't there is still an error, so check with `has` first when you're not sure. Declare everything the record will hold when you create it, using `nothing` for anything not known yet:

```
set person to {name: "Ada", age: nothing}
change person.age to 36
```

14. The 24 built-in actions

Always available. Nothing to import. **None of them ever changes what you give them** — they hand back a new value, so the original is untouched.

LISTS

ACTION	NEEDS	GIVES BACK
<code>count(thing)</code>	list, text or record	how many items, letters or fields
<code>add(list, item)</code>	a list and anything	a new list with the item on the end
<code>remove(list, position)</code>	a list and a position	a new list without that item
<code>first(list)</code>	a list with at least one item	the first item
<code>last(list)</code>	a list with at least one item	the last item
<code>reverse(list)</code>	a list or text	it, back to front
<code>has(collection, item)</code>	list, text or record	true or false
<code>sum(list)</code>	a list of numbers only	the total
<code>sort_up(list)</code>	all numbers, or all text	smallest or A–Z first
<code>sort_down(list)</code>	all numbers, or all text	largest or Z–A first
<code>sort_up(list, "field")</code>	a list of records	ordered by that field, smallest first
<code>sort_down(list, "field")</code>	a list of records	ordered by that field, largest first
<code>numbers(from, to)</code>	two whole numbers	a list counting from one to the other
<code>join(list, separator)</code>	a list and some text	one piece of text

```
set xs to [3, 1, 2]
show sort_up(xs)      # [1, 2, 3]
show sort_down(xs)    # [3, 2, 1]
show xs               # [3, 1, 2] – untouched

change xs to add(xs, 4) # to keep the result, change the name
show xs               # [3, 1, 2, 4]
```

ADD AND + ARE NOT THE SAME

`add` puts in **exactly one item**, whatever that item is. `+` **merges two lists**:

```
set xs to [1, 2]

show add(xs, 3)      # [1, 2, 3]
show xs + [3]        # [1, 2, 3] – same

show add(xs, [3, 4]) # [1, 2, [3, 4]] one new item, which is a list
show xs + [3, 4]     # [1, 2, 3, 4] four items
```

Counting makes the difference plain:

```
set names to ["Ada"]

show count(add(names, ["Alan", "Grace"])) # 2
show count(names + ["Alan", "Grace"])     # 3
```

If you're reaching for `add` with square brackets in the second slot, you almost certainly want `+`. Note that `join` is a third thing again — it

turns one list into text.

Sorting records needs the field name in quotes:

```
set people to [
  {name: "Grace", born: 1906},
  {name: "Ada", born: 1815}
]
show sort_up(people, "born") # Ada first
show sort_down(people, "born") # Grace first
```

Misspelling the field is an error naming the fields that exist, with a one-tap correction. It's checked when the line runs, not before.

TEXT

ACTION	NEEDS	GIVES BACK
<code>split(text, separator)</code>	two pieces of text	a list
<code>upper(text)</code>	text	text in capitals
<code>lower(text)</code>	text	text in lower case
<code>trim(text)</code>	text	text without leading or trailing spaces

NUMBERS

ACTION	NEEDS	GIVES BACK
<code>round(number)</code>	a number	the nearest whole number
<code>round(number, places)</code>	a number and how many places	rounded to that many decimals
<code>random(lowest, highest)</code>	two numbers	a whole number, either end possible

```
show round(17.12345, 2) # 17.12
show round(2.5) # 3
show round(-2.5) # -3 - same distance either side of zero
```

Rounding gives a number, and numbers drop trailing zeros, so `round(17.1, 2)` shows as `17.1` rather than `17.10`. Fixed decimal places for display needs text formatting, which doesn't exist yet.

CONVERTING AND RECORDS

ACTION	NEEDS	GIVES BACK
<code>text(value)</code>	anything	it as text
<code>number(text)</code>	text that reads as a number	the number
<code>keys(record)</code>	a record	a list of its field names

15. Reaching outside the program

Three actions reach beyond the program itself:

ACTION	NEEDS	GIVES BACK
<code>read(name)</code>	the name of a file	its contents as text
<code>write(name, text)</code>	a file name and some text	how many letters were written
<code>get(address)</code>	a web address	what the page returned, as text
<code>ask(question)</code>	something to ask	whatever the person types, as text

```

set raw to read("stock.csv")
set lines to split(trim(raw), "\n")
show "Lines found:", count(lines)

write("report.txt", join(lines, "\n"))

```

Two rules apply to these three and nothing else.

They need a line of their own. A kernel action can be a statement by itself, or the whole value after `set` or `change` — never buried inside a larger expression:

```

show read("stock.csv")           # no
set raw to read("stock.csv")     # yes
show raw

```

So the moment a program touches the outside world is visible on its own line, not buried in a calculation.

`ask` always gives back **text**, even when the person types a number — there's no way to know which they meant. So do sums with `number(...)`:

```

set age to ask("How old are you?")
show number(age) * 2

```

Forget it and the error says where the text came from, with a button to wrap it:

Line 2: I can only multiply numbers, but got the text "10" and the number 2. `age` holds text, because that is what `ask` gives back. Wrap it in `number(age)` to do sums with it.

They can't be used inside an action. Read at the top of your program, pass the value in, and get a value back:

```

make count_lines(raw)
  give count(split(trim(raw), "\n"))
end

set raw to read("stock.csv")
show count_lines(raw)

```

Every action you write is then free of surprises: same inputs, same result, no files touched. It also means a library written in Plain runs in a browser as happily as on a computer.

`read` only reads text — `.txt`, `.csv`, `.json` and the like. Hand it a PDF, an image or a spreadsheet and it says so rather than giving back a page of nonsense.

IN A BROWSER

A web page has no file system, so `read` asks you to pick a file and `write` hands one back as a download. `ask` is a prompt box. The program is the same either way — you just pick the file instead of naming it. `get` works, but only for sites that permit it — a limit of web pages, not of Plain.

If a file isn't chosen, or a site refuses, you get an ordinary Plain error explaining which.

16. Order of operations

From loosest to tightest:

1. `or`
2. `and`
3. comparisons — `is`, `is more than`, and the rest
4. `+` `-`
5. `*` `/` `%`

6. `not`, and negative signs
7. calls `()`, fields `.`, positions `[]`

So `2 + 3 * 4` is 14, and `a is 1 and b is 2` reads as expected. Use brackets whenever you'd rather not rely on remembering this:

```
show (2 + 3) * 4      # 20
```

17. Where names live

Two rules cover all of it.

Each block keeps its own new names. A name created inside an `if`, `for each`, `while` or `repeat` is forgotten at its `end`:

```
for each price in prices
  set found to price
end
show found
```

Line 4: `found` only exists inside the `for` on line 1. Names made inside a block are forgotten at its `end`. Create it before the block instead: `set found to nothing`

Which is exactly how to write it — create it outside, alter it inside:

```
set found to nothing
for each price in prices
  change found to price
end
show found
```

That's why a running total is always created before its loop. The loop's own name works the same way: `price` exists only for the loop.

Each action is sealed. It sees its inputs, anything it creates itself, the built-in actions, and other actions — nothing else. See section 10.

Together these give one rule worth remembering: **a name means one thing inside any single action, and one thing at the top level.** You'll almost never meet the error, because it only fires when you typed `set` where you meant `change`, or forgot a name was already in use.

18. Errors

Every error names the line, underlines the exact word, says what went wrong in a sentence, and where possible says what to do about it:

```
Line 10
show "Sum is " + totl
               ^^^^
I don't know what "totl" is.
There is something called "total". Did you mean that?
```

The full set of things Plain will tell you:

- an unknown name, with the closest name you did define (your own names are offered before built-ins)
- a name that vanished at an `end`, naming the block it was created in
- `set` on a name that already exists, and `change` on one that doesn't
- a record field that doesn't exist, with the fields that do
- a list position past the end, with how many items there are
- arithmetic on the wrong kind of value, naming both sides

- an `if` or `while` test that isn't true or false, with a suggested comparison
- an action given the wrong number of values, naming what it expects
- a block never closed, pointing at the line where it opened
- text never closed with a second quote mark
- `=` where `set` or `change` was meant, and `>` or `==` where a word was meant
- an action reaching for a name outside itself
- a field name misspelled inside `sort_up` or `sort_down`
- dividing by zero
- text that can't be read as a number
- a loop that never finishes

Where the fix is unambiguous, the playground offers it as a button.

19. The complete word list

All 29. There are no others.

Instructions — `set`, `change`, `to`, `show`, `if`, `else`, `end`, `for`, `each`, `in`, `repeat`, `times`, `while`, `make`, `give`, `stop`

Comparing values — `is`, `not`, `more`, `less`, `than`, `at`, `most`, `least`

Combining conditions — `and`, `or`

Values — `true`, `false`, `nothing`

Six of these (`more`, `less`, `than`, `at`, `most`, `least`) only ever appear as part of a comparison such as `is more than`, so in practice there are 23 words to learn and six that come along with them.

Symbols — `+`, `-`, `*`, `/`, `%`, `(`, `)`, `[`, `]`, `{`, `}`, `,`, `.`, `:`, `"`, `#`

20. What Plain does not have yet

The edges, so nothing catches you out:

- **No text formatting.** No padding, alignment or fixed decimal places, so a table of numbers comes out ragged.
- **No error handling.** No `try/catch`. An error stops the program.
- **No modules.** One program is one file; programs can't yet use each other, so there are no libraries.
- **No dates.** No way to get today's date or work out the days between two of them.
- **No input.** No way to ask the person a question while running.
- **No classes.** Records hold values, not actions on those values. This may stay that way deliberately.
- **No async, threads or timing.** Nothing runs in the background.

SETTLED SINCE V0.1

Every rough edge found so far is now closed:

1. **Names vanishing at `end`** — kept, because block scoping is correct, but the error names the block and says what to do instead.
2. **Mutation** — settled completely. Nothing is changed in place, and every binding takes a copy, so no two names ever share a list or record.
3. **Suggestions favouring built-ins** — fixed. Your own names are offered first.
4. **Asymmetric rounding** — fixed. `round(2.5)` is 3 and `round(-2.5)` is -3.
5. **Runaway recursion** — an action may call itself 300 times, then stops with a message naming it. Nothing internal can leak, even on a device with less room than usual.
6. **Silent flooring** — `repeat`, list positions and `random` all require whole numbers.
7. **Fields appearing from a typo** — `change` can no longer invent a field.
8. **Actions inside actions** — no longer allowed; every action lives at the top level.
9. **No way to touch the outside world** — `read`, `write` and `get` now exist, confined to statements outside any action.

OPEN QUESTIONS

- **Checking earlier.** Quoted field names are checked when the line runs. A pass over the whole program before running would catch them sooner, along with unknown names and wrong argument counts. Planned for after the kernel.
- **Text formatting.** The shape of it isn't decided yet — likely a small group of actions rather than one.
- **Copying cost.** Every binding copies, which is $O(n)$ for a large list. Copy-on-write would remove the cost without changing anything observable, if it ever matters.

21. A complete program

Everything in this document, used once:

```
# Work out which product line earned most last month.

set lines to [
  {name: "Monitors", units: 42, price: 189.99},
  {name: "Keyboards", units: 118, price: 34.50},
  {name: "Cables", units: 306, price: 4.25}
]

make revenue(line)
  give line.units * line.price
end

set totals to []

for each line in lines
  set earned to revenue(line)
  change totals to add(totals, earned)
  show line.name + ": " + round(earned, 2)
end

show ""
show "Total:", round(sum(totals), 2)
show "Average per line:", round(sum(totals) / count(lines), 2)

show ""
show "Best first:"
for each line in sort_down(lines, "units")
  show "  " + line.name + " — " + line.units + " units"
end
```

```
Monitors: 7979.58
Keyboards: 4071
Cables: 1300.5

Total: 13351.08
Average per line: 4450.36

Best first:
  Cables — 306 units
  Keyboards — 118 units
  Monitors — 42 units
```

Note the shape: `set totals to []` before the loop, `change` inside it, and `revenue` taking everything it needs as an input.

22. Everything, in alphabetical order

Every word and every built-in action, with where to read more.

WORDS

WORD	WHAT IT DOES	SECTION
<code>and</code>	Both conditions must be true	9
<code>at least</code>	Bigger or the same (part of <code>is at least</code>)	8
<code>at most</code>	Smaller or the same (part of <code>is at most</code>)	8
<code>change</code>	Give an existing name a new value	4
<code>each</code>	Part of <code>for each</code>	10
<code>else</code>	The other branch of an <code>if</code> . Needs its own line	9
<code>end</code>	Closes a block. Needs its own line	1
<code>false</code>	One of the two things an <code>if</code> accepts	2
<code>for</code>	Start of <code>for each</code>	10
<code>give</code>	Hand a value back from an action, and stop it there	11
<code>if</code>	Do something only when a condition holds	9
<code>in</code>	Part of <code>for each ... in</code>	10
<code>is</code>	Exactly the same. There is no <code>==</code>	8
<code>is at least</code>	Bigger or the same	8
<code>is at most</code>	Smaller or the same	8
<code>is less than</code>	Smaller	8
<code>is more than</code>	Bigger	8
<code>is not</code>	Different	8
<code>less</code>	Part of <code>is less than</code>	8
<code>make</code>	Create an action. Top level only	11
<code>more</code>	Part of <code>is more than</code>	8
<code>nothing</code>	No value yet	2
<code>not</code>	Turns true into false	9
<code>or</code>	Either condition may be true	9
<code>repeat</code>	Do something a fixed number of whole times	10
<code>set</code>	Create a name. Fails if it already exists	4
<code>show</code>	Display something. Commas put several on one line	5
<code>stop</code>	Leave the innermost loop early	10
<code>than</code>	Part of <code>is more than</code> and <code>is less than</code>	8
<code>times</code>	Part of <code>repeat ... times</code>	10
<code>to</code>	Part of <code>set ... to</code> and <code>change ... to</code>	4
<code>true</code>	One of the two things an <code>if</code> accepts	2
<code>while</code>	Keep going until a condition stops holding	10

ACTIONS

ACTION	WHAT IT DOES	SECTION
<code>add(list, item)</code>	A new list with one more item on the end	14
<code>ask(question)</code>	Ask the person something. Always gives text	15
<code>count(thing)</code>	How many items, letters or entries	14
<code>first(list)</code>	The first item	14
<code>get(address)</code>	Fetch a web address, as text	15
<code>has(thing, item)</code>	Whether it's in there. True or false	14
<code>join(list, separator)</code>	One list into one piece of text	14
<code>keys(record)</code>	A list of a record's names	14
<code>last(list)</code>	The last item	14
<code>lower(text)</code>	Text in lower case	14
<code>number(text)</code>	Text into a number. Fails if it isn't one	14
<code>numbers(from, to)</code>	A list counting from one to the other	14
<code>random(low, high)</code>	A whole number between the two, either end possible	14
<code>read(name)</code>	Read a text file	15
<code>remove(list, position)</code>	A new list without that item	14
<code>reverse(list)</code>	It, back to front. Works on text too	14
<code>round(number)</code>	Nearest whole number	14
<code>round(number, places)</code>	Rounded to that many decimals	14
<code>sort_down(list)</code>	Largest or Z-A first	14
<code>sort_down(list, "field")</code>	Records ordered by a field, largest first	14
<code>sort_up(list)</code>	Smallest or A-Z first	14
<code>sort_up(list, "field")</code>	Records ordered by a field, smallest first	14
<code>split(text, separator)</code>	One piece of text into a list	14
<code>sum(list)</code>	The total of a list of numbers	14
<code>text(value)</code>	Anything, as text	14
<code>trim(text)</code>	Text without spaces at either end	14
<code>upper(text)</code>	Text in capitals	14
<code>write(name, text)</code>	Write a text file	15

The four in section 15 reach outside the program. Each needs a line of its own and none can be used inside an action.